Timed SCCharts versus Multiform Time in a Traffic Light Controller

Harold Tien (thar970) Lucas Young (wyou790)
COMPSYS 303 Assignment, Group 14
Department of Electrical, Computer, and Software Engineering
University of Auckland, New Zealand

***Abstract*—We built a four-task traffic light controller in SCCharts twice. Version A uses Timed SCCharts clocks and runs in the KIELER simulator with dynamic ticks. Version B has no clocks and counts a 10 ms Nios II timer event on a DE2-115 board. Apart from their timing statements and a six-line camera wake-up block, the two charts are identical, and they give the same light sequences. Version A meets every time-out exactly in simulated time with about six reactions per 17 s cycle. With a 10 ms timer, Version B keeps the Task 1 phases within about 0.5 ms on the board, but it rounds time-outs up to whole timer periods and computes about 37,000 reactions per cycle, of which only 1,700 carry a timer event.**

I. Introduction

The brief asks for a controller for two crossing roads, built up in four tasks: a six-phase cycle (Task 1), pedestrian lights (Task 2), run-time configuration (Task 3) and a red-light camera (Task 4). All charts are written in SCCharts [1]. In Version A every time-out is a Timed SCCharts clock compared directly with a time value [2], [3]. In Version B the chart has no notion of time and counts timer interrupts from the Nios II, which is the multiform view of time of the classic synchronous languages [4].

We wrote one chart that holds Tasks 1–4, once as Version A and once as Version B. All measured numbers come from the traces and board logs in the `evidence` folder of our submission.

II. Controller Design

A. Interface and Task 1

The combined chart reads `task` (SW1..0 on the board), `NSraised` and `EWraised` (KEY0, KEY1), `vehicleButton` (KEY2), one Task 3 character per tick (`configChar`, `configValid`) and, in Version B only, `timerTick`. Its output `LEDR` follows Table 1 of the brief, with bits 6 and 7 for the NS and EW walk lights, so only seven words can appear: 33 (G-R), 34 (Y-R), 36 (R-R), 12 (R-G), 20 (R-Y), and 97 or 140 (a green with its walk light). Each phase of the cycle is one state of the Lights region (GR, YR, RR1, RG, RY, RR2; Fig. 1). Its entry action resets the phase timer `c` and writes the whole `LEDR` word, so only Lights writes the lamps (a Moore output), and a yellow state clears a walk bit just by rewriting the word. The exit guard compares `c` with the active time-out $t_1..t_6$. The defaults (T1..T6 = 6, 2, 0.5, 6, 2, 0.5 s), `WALK_TIME` = 4 s and `CAMERA_DWELL` = 1 s are named constants at the top of the charts that use them. The active values $t_1..t_6$ are separate variables because Task 3 overwrites them.

The exit from each all-red decision chain (Section II-C) must be immediate so that the green starts in the tick the all-red ends, so the one delayed transition of the cycle sits on GR→YR. A green always spans more than one tick, so this costs nothing, and GR can never be left in the tick it was entered.

B. Task 2: pedestrian lights

Two request regions run concurrently with Lights (Fig. 2). NSRequest increments `nsRequests` on a press and waits in `PressedNS` until the button is released, so a held button counts once. Lights keeps its own counter `nsServed` and sees a pending request when `pre(nsRequests) > nsServed`. Each shared variable has one writer, and Lights reads the request count one tick after it was written, as the brief suggests. The data flows

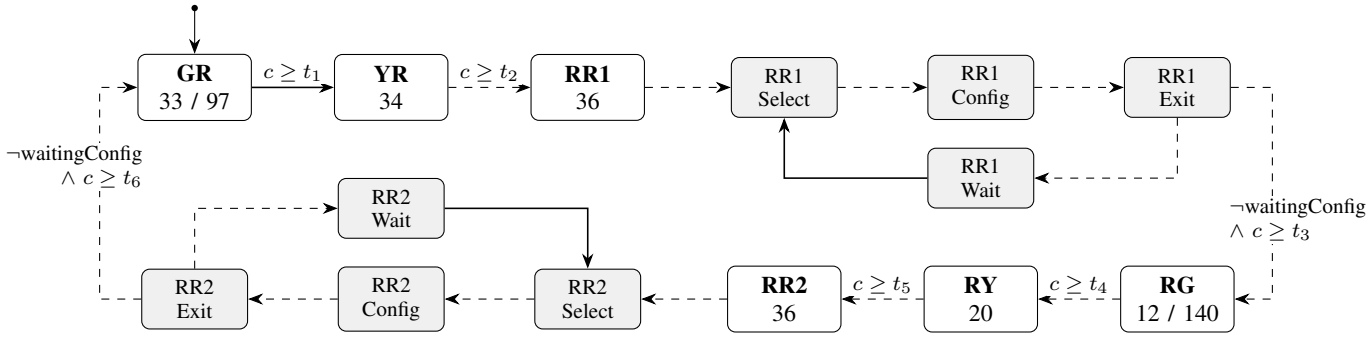


Fig. 1. Lights region of the combined chart, the same in both versions (dashed: immediate, solid: delayed, as in KIELER). GR and RG contain the walk sub-regions (Fig. 2). Only RRnSelect copies `task` into `activeTask`; RRnConfig applies a waiting Task 3 packet. In Version B each guard reads $c \cdot \texttt{TIMER_MS} \geq t_n$.

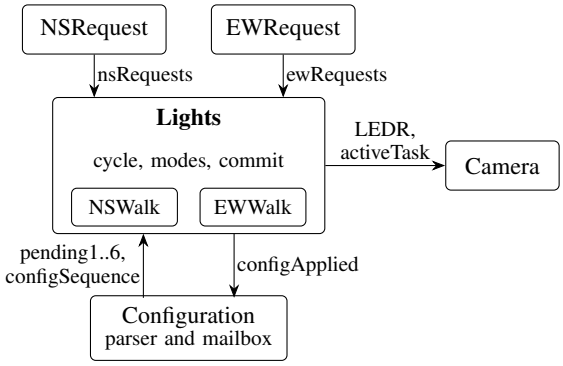


Fig. 2. Regions of the combined chart and the variables they share. Every variable has a single writer. NSWalk and EWWalk sit inside GR and RG. Lights and Configuration read each other's variables with `pre()`; Camera reads `LEDR` in the same tick.

one way only, and serving a request is just `nsServed = pre(nsRequests)`.

The walk is a sub-region inside each green state. On entry to GR, `DecideNS` checks for a pending request (only if `activeTask` ≥ 2), shows 97 for `WALK_TIME` on its own timer `wNS`, and then 33 for the rest of the green, which still lasts t_1. Leaving GR ends the walk, so a walk light can never reach yellow, even if Task 3 makes the green shorter than 4 s. A press during its own green is served at the next green (a press while the walk light is on is absorbed by that walk), and requests are kept when the task changes.

C. Safe mode switching

The input `task` is read only in RR1Select and RR2Select, which are entered immediately after an all-red state has written 36 (Fig. 1). RRnSelect copies `task` into `activeTask` (Task 3 also sets `waitingConfig`), RRnConfig applies a waiting Task 3 packet, and RRnExit starts the next green if the controller is not waiting for a packet and the all-red time has passed. Otherwise the delayed RRnWait→RRnSelect repeats the decision in the next tick. All task-dependent logic reads `activeTask`, so a task change requested in the middle of a green takes effect at the next all-red (tests A6 and C5.1), and invalid values are ignored. If the all-red time has already passed when the task changes, for example when leaving a Task 3 hold, the next green starts in the same tick, so `activeTask` only changes on a tick that starts or ends at 36.

Every loop has exactly one delayed transition: GR→YR for the cycle, RRnWait→RRnSelect for each all-red loop, Pressed→Released in the request regions, NextCharacter→Receive and Published→Receive in the parser, and NextCameraTick→SampleVehicle (plus ArmedTick→SampleVehicle in Version A) in the camera. Besides breaking instantaneous loops, these edges limit each tick to one parsed character and one camera decision.

D. Task 3: run-time configuration

The Configuration region (states Receive, Classify, NextCharacter and Published) parses one ASCII character per tick. It accepts six comma-separated fields of one to four digits ended by LF, and ignores CR. Anything else (an empty field, a fifth digit, a seventh field, a letter) makes the packet invalid, and it is dropped when its LF arrives. A valid packet goes into a one-slot mailbox, and `configReady` stays false until Lights acknowl-

edges it, so the sender (`main.c` on the board) only sends while `configReady` is true. Lights reads the mailbox through `pre()` and copies all six values at once in RRnConfig, so time-outs only change in R-R. The commit also restarts `c`, so the new all-red is held in full.

The brief gives no unit, and we chose milliseconds (1 to 9999). In seconds the default 0.5 s all-red could not be sent, and one field could mean almost three hours. We also reject 0: a random test of an earlier parser that accepted 0 produced a green followed directly by the other green (33→140). Entering Task 3 holds the lights at all-red until a valid packet arrives. Task 4 also applies packets.

E. Task 4: red-light camera

The Camera region (SampleVehicle, Report-Leave, CheckCamera, NextCameraTick) toggles `vehiclePresent` on each rising edge of `vehicleButton`. If a car is present while a direction is yellow (`LEDR` is 34 or 20) and has not been photographed on this visit, the camera resets its own dwell timer and prints "Camera activated". If the car is still there after `CAMERA_DWELL` it prints "Snapshot taken", and a departure prints "Vehicle left" and cancels the timer. The departure is checked before the deadline in the same tick, so a car that leaves exactly on the deadline is not photographed. We read the brief as arming the camera only for a car that is in the intersection during yellow, so a car that arrives during yellow starts its dwell on arrival. Camera reads `LEDR` and `activeTask` but never writes them.

III. Timing Mechanisms

A. Version A: clocks and dynamic ticks

Version A has four clocks: `c`, the walk clocks `wNS` and `wEW` (declared inside their green states) and `cameraClock`. A clock is reset when its activity starts, and the guard reads as a time-out, for example `c >= t1`. KIELER compiles each clock into a variable that is advanced by the elapsed time `deltaT` at each tick, and the generated code returns `sleepT`, the time until the nearest clock guard can become true [3]. In Dynamic mode the simulator uses `sleepT` as the next `deltaT`, so the chart only reacts when a time-out expires or an input changes.

TABLE I
Differences Between the Two Combined Charts

	Version A	Version B
Time-outs	seconds (`float`)	milliseconds (`int`)
Time variables	4 clocks	4 counters, 9 increments
Guard	`c >= t1`	`c*TIMER_MS >= t1`
Time input	`deltaT`	`timerTick`
Extra states	ArmedCheck/Tick	none
Task 3 commit	packet / 1000	packet as sent

There is a trap here. `sleepT` only comes from guards that are evaluated in the current tick; otherwise it keeps its default of 1000 s, which means no wake-up. When the camera was armed, our chart reset `cameraClock` and went straight to a delayed state, so in Dynamic mode the snapshot slipped to the next light change. The extra state ArmedCheck evaluates the dwell guard in the arming tick and so requests a wake-up 1 s later. The same happened after a Task 3 commit, which now goes through RRnExit for the same reason. Version B ticks all the time and does not rely on either change.

B. Version B: counting timer events

Version B has no clocks. A boolean input `timerTick` is true on a tick that carries one timer event, and `TIMER_MS = 10` is the only named rate in the chart (the chart cannot read `main.c`, which keeps a matching `TIMER_PERIOD_MS` for the real alarm). Each timed activity has a counter (`c`, `wNS`, `wEW`, `camCount`), and a guard such as `c * TIMER_MS >= t1` holds after $\lceil t_1/\texttt{TIMER_MS} \rceil$ events. Where the count goes matters. States that rest for many ticks (GR, YR, RG, RY and the walk states) count in a non-immediate `during` action, which skips the entry tick, so the event that ended the previous phase is not counted again. The all-red loop and the camera loop are re-entered in every tick, so a `during` action there would never run, and they count on their delayed edge instead. `c` and `camCount` count the same event stream for two purposes, as the brief describes for Task 4 (Table I lists all differences).

In `main.c`, a HAL alarm callback runs inside the 1 ms `timer_0` interrupt once every `TIMER_PERIOD_MS` and only increments `pendingEvents` [5]. The main loop follows the

Sensor–Controller–Plant skeleton of the brief. It takes at most one pending event (with interrupts disabled for the decrement), reads the keys, the switches and at most one UART byte, calls the generated `tick()`, and writes `LEDR` to the LEDs. If the loop is slow, events wait in the counter instead of being lost. `timer_1` runs as a 50 MHz timestamp [6] and only measures durations for this report. The loop runs freely at about 2,140 to 2,190 ticks per second, one pass every 0.46 ms.

IV. Version A versus Version B

A. Method

Version A was run with KIELER's Netlist-based Simulation (C), built from the submitted chart and driven by a script that reproduces Dynamic mode: a tick happens when `sleepT` is due or when an input changes. All 13 cases pass (A2–A11, A13–A15, as in `final_test_results.md`). The strict runs (A8, A14, A15) use only these ticks, plus one at the end of each scripted run; the others also tick every 0.1 s when no wake-up is requested, and A2 runs in real time on the PC clock. Version B ran on the DE2-115 with the same time-outs and scenarios, and `main.c` logs each `LEDR` change with the length of the phase that ended and its tick and event counts. The generated Version B code also passed 30 PC test cases, two of them with 10^6 random ticks, and a checker found no illegal LED word or transition in the 22 board logs (2,172 phases).

B. Correctness and equivalence

Both versions give the same sequences. Task 1 cycles 33, 34, 36, 12, 20, 36. A pedestrian request gives 97 for 4 s and then 33 for 2 s; on the board every walk was exactly 400 events (3999.7 to 4000.4 ms). With a car present at yellow both versions print "Camera activated" at the start of the yellow, "Snapshot taken" 1 s later (Version A 1.000 s, Version B 100 events, measured as 998.9 to 999.5 ms) and "Vehicle left" when it leaves, and a car leaving within 1 s is not photographed. The packet `8000,3000,1000,8000,3000,1000` gives 8/3/1 s in Version A and 800/300/100 events (7999.8, 2999.9 and 1000.2 ms) in Version B, and invalid packets are rejected by both. A script that removes the timing statements finds the two charts identical, apart from the ArmedCheck/ArmedTick block.

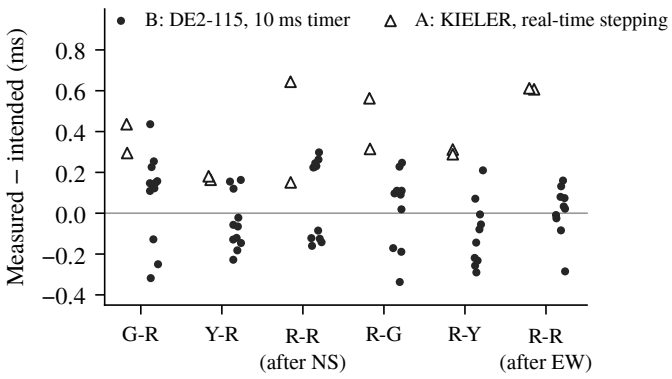


Fig. 3. Phase error in the default Task 1 cycle. Version A (run A2, 12 phases): sum of measured host sleeps, simulator run time excluded. Version B (63 phases): board timestamps, 10 ms timer. Different timing bases: not an end-to-end accuracy ranking.

Timing can still differ. Version B ends a time-out only on a timer event, so a time-out T lasts $\lceil T/P \rceil P$ for a timer period P, and an activity that starts between two events sees a partial first period and can end up to one period early. Version A has neither effect.

C. Timing accuracy and jitter

In simulated time Version A meets every time-out exactly, because the simulator advances by exactly the `sleepT` the chart asked for (run A14: every phase exact over 170 s). In real time it is limited by how late the host wakes up. Stepping the simulator on the PC clock (A2), phases timed as the sum of the measured sleeps were 0.15 to 0.64 ms late and never early (the simulator's own run time is not included) (Fig. 3).

At 10 ms Version B counted exactly 600/200/50 events in every phase. Mean durations were within 0.10 ms of nominal, single phases were between -0.34 and $+0.44$ ms, and the largest spread for one phase type was 0.75 ms (Table II). A 15-minute Task 4 run with no car (324 phases) stayed within -0.41 to $+0.43$ ms, and its 918.0 s total equals 91,800 events $\times$ 10 ms. This is not quantisation, because phases start and end on timer events. It is loop latency (0.46 ms is the mean loop pass, not a worst-case bound) plus the time spent in `printf`.

The period matters when a time-out is not a multiple of it. At 1, 10 and 100 ms all defaults

TABLE II
VERSION B TASK 1 TIMING AGAINST THE TIMER PERIOD

P (ms)	Events	G-R / Y-R / R-R (ms)	Err.	Jitter
1	6000/2000/500	6000.0 / 1999.9 / 500.0	0.13	1.19
10	600/200/50	6000.1 / 2000.0 / 500.1	0.10	0.75
100	60/20/5	6000.0 / 2000.0 / 500.1	0.19	0.81
300	20/7/2	6000.2 / **2099.9** / **600.0**	0.22	0.53
10/20*	600/200/50	**12000.3** / **3999.9** / **1000.0**	0.30	0.34

Means of the NS phases. Err. = worst mean error, Jitter = largest max−min, over all six phases, in ms, against $\lceil T/\texttt{TIMER_MS} \rceil$ events of the real period. *Chart 10 ms, `main.c` 20 ms.

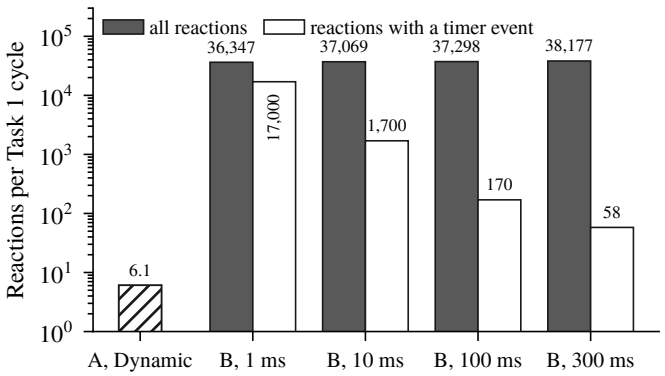


Fig. 4. Reactions per Task 1 cycle (log scale). Version A (hatched): strict Dynamic mode, run A14. Version B: board logs at four timer periods (at 300 ms a cycle is 17.4 s because of rounding).

divide exactly. At 300 ms the yellow becomes $\lceil 2000/300 \rceil = 7$ events = 2100 ms and the all-red 600 ms, while the green stays 6000 ms. The error is always on the long side, because the guard uses $\geq$. Changing only `TIMER_PERIOD_MS` in `main.c` to 20 ms doubled every phase with the same event counts: the chart only counts events. Alignment gives the other kind of error. At P = 100 ms a camera armed at the start of a yellow measured 999.1 to 999.3 ms, but a car arriving during the yellow 936 to 960 ms, and at 10 ms the first green after leaving a Task 3 hold lasted 5997.2 ms. (Our dwell stopwatch starts after the log line is printed, so dwell readings are about 0.8 ms short.)

D. Computational effort

In Dynamic mode Version A needed 61 reactions for 10 Task 1 cycles, one per light change plus one early wake-up after the first tick (Fig. 4). For about two Task 4 cycles with a car it needed 19: 14 wake-ups, 4 button ticks and 1 tick added by the test script. Version B computed 37,067 to 37,072 reactions per cycle at 10 ms, and only 1,700 carried a timer event, so 95.4% of them carried none. Task 4 costs it the same (37,056 per cycle with no car, about 37,030 with a car in run C4). A slower timer only raises the idle share (53% at 1 ms, 99.5% at 100 ms). Even a Version B that ticked only on timer events would need 1,700 reactions per cycle, about 280 times more than Version A, and Version A with a fixed 10 ms tick would need the same.

The difference is what each version waits for. Version B has to see every timer event to count it, so its work grows with elapsed time. Version A only reacts when a clock reaches a guard or an input arrives, so its work grows with the number of events. During a Task 3 hold Version A does nothing until an input arrives, while Version B keeps ticking over 2,000 times a second. The cost moves into the runtime, which has to honour `sleepT`, as in [2].

E. Modelling clarity

Version A reads better, and for Task 4 it is clearly simpler: `c >= t1` says what it means, and the camera is just one more clock and guard. In Version B the same event is counted twice in Task 4, and for each state we had to decide whether the count belongs in a `during` action or on a delayed edge. A wrong choice either counts the event that ended the previous phase twice or never counts at all (RRnWait). The time base is also split over two files: `TIMER_MS` in the chart must equal `TIMER_PERIOD_MS` in `main.c`, and the 10/20 row of Table II shows what happens otherwise.

Version A hides its own complexity. Which guards KIELER evaluates in a tick is not visible in the chart, both of its timing bugs only appeared in Dynamic mode, and the camera fix needed two states that mean nothing for the traffic light. Version B has no hidden scheduler and every counter is always up to date. Overall Version A was easier to design and read, and Version B easier to reason about once the counting rules were settled.

F. Reconfiguration (Task 3)

The parser is the same in both versions, the commit differs only in the unit conversion, and both restart the all-red timer on a commit. Version A uses a new value exactly (divided by 1000), but

a new time-out also changes when the chart has to wake up. Our first commit did not request a wake-up, so in strict Dynamic mode the following all-red lasted until the next input. Going through RRnExit, the commit tick requests `sleepT` $= t_3$, and the all-red ended exactly 1.0 s later (A8). Version B cannot lose a deadline this way, because it counts the next event anyway, but it rounds every value up: `1234,567,89,4321,765,98` gave 124/57/9/433/77/10 events, so the 1234 ms green lasted 1240.1 ms. Two packets sent back to back were both committed in one all-red, and the second commit restarted the count, so that all-red lasted 101 events instead of 100 (the safe direction). So reconfiguration was harder to get right in Version A and less precise in Version B.

Summary. Every difference above comes from how time is represented. Version A treats time as a quantity, so time-outs are exact and the chart runs only when something is due, but the runtime must schedule each wake-up. Version B rebuilds time by counting an event, so its accuracy is set by the event period and alignment, and it reacts to every event.

V. Difficulties and Future Work

KIELER. Both charts compile with "Instantaneous loop detected!", although every loop has a delayed transition. Four small test charts showed that the warning comes from KIELER's expansion of `during` actions and clocks: a plain delayed loop compiles silently, and only an all-immediate loop is rejected. KIELER also gave both transitions out of ArmedCheck the same guard in the generated code, so the camera region stopped after "Camera activated" (tests A10, A11). Sending the second transition to a separate state, ArmedTick, avoided it.

Parser. A bug dropped the first character after reset, because `pre(configReady)` is false in the first tick; reading `configReady` directly fixed it.

Board. Quartus Lite has no licence for the Nios II/f core, so we only have a time-limited `.sof` that needs an active OpenCore Plus JTAG link. Our stopwatch froze after 85.9 s until we set the timer's CONT bit, as the HAL starts it in one-shot mode.

Future work. Run Version A on the board with a one-shot timer set to `sleepT`, let Version B sleep until an interrupt, use one chart with a timing switch, boot from flash, and check the safety property with a synchronous observer [4].

TABLE III
Hours per Part and Workload

Hours per part	Harold (M1)	Lucas (M2)	Total
Design (states, regions)	3	3	6
Task 1	2	2	4
Task 2	3	3	6
Task 3	4	4	8
Task 4	3	3	6
Version A (clocks, KIELER)	3	3	6
Version B (counting, board)	5	5	10
Comparison and experiments	3	3	6
Documentation and report	3	3	6
Workload: time spent (hrs)	29	29	58
Design / logic (%)	50	50	
Implementation (%)	50	50	
Debugging & simulation (%)	50	50	

Percentages are each member's share of that kind of work.

VI. Time Spent and Workload

Table III lists our hours and workload.

Use of Generative AI

Harold and Lucas designed the system and made all design decisions. We discussed the structure of the code with Claude (Anthropic) and OpenAI Codex, which helped write parts of the charts, `main.c` and the scripts; Claude also helped draft this report, and ChatGPT (OpenAI) was used for parts of the work. Each source file has its own declaration. We verified all of it against our traces and logs.

References

[1] R. von Hanxleden *et al.*, "SCCharts: Sequentially constructive statecharts for safety-critical applications," in *Proc. 35th ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, Edinburgh, U.K., 2014, pp. 372–383.

[2] A. Boysen, A. Schulz-Rosengarten, and R. von Hanxleden, "A hard real time demo for dynamic ticks and Timed SCCharts," Kiel Univ., Kiel, Germany, Tech. Rep.

[3] A. Schulz-Rosengarten, R. von Hanxleden, F. Mallet, R. de Simone, and J. Deantoni, "Time in SCCharts," in *Proc. Forum Specification Design Lang. (FDL)*, 2018.

[4] A. Benveniste, P. Caspi, S. A. Edwards, N. Halbwachs, P. Le Guernic, and R. de Simone, "The synchronous languages 12 years later," *Proc. IEEE*, vol. 91, no. 1, pp. 64–83, Jan. 2003.

[5] Intel Corp., *Nios II Software Developer's Handbook*, NII-SDH, Oct. 2018.

[6] Intel Corp., *Embedded Peripherals IP User Guide*, UG-01085, Sep. 2018.